

Handoff Memo: Two Gemma-4-E4B LoRA Adapters for Exquisite Corpse

Fine-tuning plan for an always-on vLLM host: a next-line writing adapter and a close-reading adapter

Prepared September 12, 2026 - Zach Muhlbauer (CUNY GC) - repo: cadavre-exquis (master) - this document is the plan of record; execute in the order written.

1. Purpose and current state

This memo specifies the engineering work required to split the Exquisite Corpse game's language-model duties across two fine-tuned adapters served from one always-on vLLM host, and it records the decisions already taken. The game as deployed asks a single model to do two distinct jobs. During play it supplies the corpse's next contribution - one or two words extending the human's cue, under the locked game system prompt - and when a player types a single period it instead writes the close reading of the finished poem: two to four concrete sentences that quote the poem's exact words. Both UIs (the parlor at [index.html](#) and the open sheet at [ui/corpse.html](#)) address one model name today, so both tasks are performed by the same weights. The plan replaces that single adapter with two specialized ones - `exquisite-corpse-write` and `exquisite-corpse-read` - trained on the same base model so they can co-serve from the existing multi-LoRA vLLM host alongside `cloze-reader` and `jeopardylm`.

The repo is in unusually good shape for this work. The cleaned corpus stands at 21,834 unique public-domain poems, yielding 247,038 train and 9,377 validation next-line examples; the regression suite is 49 tests and green; a shipped surrealist-continuation adapter already exists on the Hugging Face hub; and the quantitative harness, `train/eval_nll.py`, measures teacher-forced next-line NLL through the live host without touching the GPU. The known follow-up #8 - retrain the writer on the current, cleaned corpus - folds naturally into this work as the write adapter. The machine is a 24-core box with 62 GB of RAM and an RTX 5090 (32 GB); the host has been up six days under `systemd` as a user unit with `Restart=always`, which is exactly the always-on property we need to preserve.

2. Decisions locked with the operator

- Base model: both adapters train on `unsloth/gemma-4-E4B-it`, the same base the existing `cloze-reader` and `jeopardylm` adapters use. A LoRA only loads on the base it was trained on, so this keeps one vLLM host, one port (`:1234`), and the proven Blackwell serving recipe.

- Reading data authorship: a frontier teacher model, called through a provider API with a key the operator supplies, writes the close-reading paragraphs. This mirrors the cloze-reader precedent of Gemma+Gemini distillation. The key lives only in a gitignored environment file.
- GPU access: one maintenance window of roughly four to six hours, inside which the vLLM host is stopped, both adapters are trained back to back, and the host is restarted and re-verified. A reboot precedes the window to clear a known NVML driver/library mismatch so nvidia-smi can monitor the runs.
- Concurrency target: the host is provisioned for at most two simultaneous requests. This is well inside what one E4B base with LoRA modules and continuous batching can absorb; no second server is needed.

3. Write adapter (exquisite-corpse-write)

The writer's data problem is already solved: `src/build_dataset.py` windows each poem into next-line chat examples with model-agnostic messages, splits by poem so nothing leaks across the split, and down-weights the Gutenberg Poetry Corpus padding to one half of the modernist core. The training data is therefore the current cleaned corpus, regenerated in place. What changes is bookkeeping rather than recipe: train via the `scripts/retrain.sh` flow, which verifies the regression suite, pins the validation fingerprint (the report embeds the sha256 of the exact val bytes, currently 481b0a3dd556), archives the shipped adapter to `outputs/lora-shipped-20260715` before `outputs/lora` is overwritten, stops the host and waits for VRAM to release, trains the E4B recipe (rank 16, loss masked to the assistant line, 2,500 steps), and restarts the host. For this project the output directory is targeted at `outputs/lora-write` so the legacy adapter name keeps serving until cutover. The gate to beat is explicit: tuned NLL per token below 3.6581 on the pinned validation set, measured with the same 400-example, seed-11 command. The shipped adapter's current gap over base is 2.27 nats, about nine point six times the per-token likelihood, so the retrain is expected to hold or improve that number given the removed wrapped-prose noise.

4. Read adapter (exquisite-corpse-read) and its dataset

The reader needs data that does not yet exist: poems paired with close readings in the exact voice the game already demands. A new module, `src/build_reading_dataset.py`, samples from the cleaned interim corpus - public domain only, per the corpus lock - stratified across the surreal and modernist core and the broader padding. Each item is either a mid-poem excerpt of two to fourteen lines (about sixty percent of items, so the model learns to read fragments the

game actually produces) or an entire short poem (about forty percent). For every item the teacher model receives the verbatim close-reading prompt the UI uses - one paragraph, two to four concrete sentences, quoting the poem's exact words, no mention of the game or the turns, no generalizing about Surrealism, no praise, no theme assignment - and produces best-of-k candidates from which one is accepted. Style anchors are few-shot exemplars drawn from public-domain critical prose already in scope: Poe's editorial criticism, Eliot's The Sacred Wood, Arnold and Bradley. Anchors condition style; they are not training pairs and are not emitted as assistant turns.

Every accepted pair must pass deterministic validators before it enters the dataset: sentence count between two and four; at least one quoted span that appears verbatim in the source lines; none of the forbidden meta references (game, turns, corpse, surrealist movement); no praise or theme cliches from a blocklist; a length cap in tokens; and a near-duplicate check across the dataset by normalized-text hash. Items failing generation are retried, and poems whose candidates never pass are dropped. The target size is six to ten thousand validated pairs, split ninety-five to five by poem, with a dataset card and a private push to the Hub mirroring `deploy/push_hf_dataset.sh`. The training script, `train_read_qlora.py`, mirrors `train_qlora.py` exactly - same Unsloth QLoRA recipe, rank 16, response-masked loss - with the sequence length raised to about 1,536 tokens so entire poems fit in context.

5. Serving: one host, four adapters, always on

Serving changes are deliberately small. `scripts/vllm_serve.sh` gains two `--lora-modules` entries, `exquisite-corpse-write` and `exquisite-corpse-read`, while the legacy `exquisite-corpse` name keeps pointing at the current adapter during migration, so nothing breaks before the UIs are switched. `--max-loras` rises from three to four. The VRAM arithmetic is comfortable: each rank-16 E4B LoRA costs on the order of 150 to 250 megabytes of resident weight and activation overhead against a 32-gigabyte card already budgeted at ninety percent utilization for the base and KV cache; four adapters remain well inside the margin, and the operator should watch the host's first-boot log for the LoRA warm-up line to confirm. Always-on status is consolidated on the existing systemd user unit, `inference-arcade-vllm.service`, whose `Restart=always` already does the job; the unit's `ExecStart` points at `serve_gemma.sh`, which must gain the same two adapter entries. Concurrency of two simultaneous requests is handled by vLLM's continuous batching natively - the host already serves three games from one base - so the capacity statement is that the box serves the whole arcade at two concurrent requests with headroom, and the only tuning knob likely to matter is `--max-num-seqs` if latency at concurrency two ever disappoints.

6. UI cutover

The two UIs separate the two prompts already; the change is which model name each call carries. `ui/config.local.js` gains a `modelRead` field next to `model`, defaulting to the write adapter, with the read adapter named for reading calls. In the open sheet, `corpse.html`, the single-period flow becomes two calls: the accumulated contributions are replayed locally as today, and the close-reading request goes to the read adapter. In the parlor, `index.html`, the game turn call and the `readingPrompt` call simply point at their respective names. A small compatibility shim keeps the legacy single-model configuration working if `modelRead` is absent, so a stale config degrades gracefully rather than breaking play. The cutover is reversible by reverting one config file.

7. Evaluation gates and acceptance

Acceptance is defined by numbers, not impressions. The write adapter must beat NLL per token of 3.6581 on the pinned validation set, measured by `train/eval_nll.py` with 400 examples at seed 11 through the live host; the report embeds the val fingerprint, and any drift aborts the comparison. The read adapter gets a new `train/eval_read.py` computing, on held-out items: a structural pass rate against the same validators used at data build time (target at least ninety percent), a quote-faithfulness rate (every quotation in the output must appear verbatim in the input poem; target at least ninety-five percent), and teacher-forced NLL of held-out gold readings. Qualitatively, a sheet of a dozen random poems per adapter, generated through the live host, is filed under `docs/` for the operator to read. The regression suite must remain green (currently 49 tests), and new validators get unit tests in the same style - red first, then the rule - as the existing pipeline tests.

8. Risks and mitigations

Four risks are worth naming. Quote hallucination: the reader may quote words the poem does not contain; the validator blocklist at build time plus the quote-faithfulness gate at eval time bound this, and the generation prompt instructs quoting exactly. Val drift: the retrain comparison is only meaningful against the pinned fingerprint; `retrain.sh` already aborts if the val set moved, and `FORCE=1` is the documented override. VRAM: the fourth LoRA is the new resident cost, and the mitigation is the margin calculation above plus the ability to drop the legacy slot after cutover. Teacher API availability: generation is resumable and checkpointed per item like the existing fetchers, so an interrupted run resumes rather than restarts, and a later key upgrade can regenerate a second dataset version without touching the recipe. One housekeeping risk is recorded rather than mitigated: this box currently has an NVML driver/library mismatch, so a reboot before the training window is part of the schedule, not an

optional step.

9. Command sequence for the maintenance window

- Reboot the box; on return, confirm nvidia-smi reports the RTX 5090 and the systemd unit has restarted the host on :1234.
- Set `TEACHER_API_KEY` and `provider` in the `gitignored env` file; run `src/build_reading_dataset.py` to produce `data/processed/reading.train/val.jsonl` with the validators' pass-rate summary.
- Stop the host (`scripts/retrain.sh` does this internally); train the write adapter on the cleaned corpus per section 3; run `eval_nll.py` and record the number against the 3.6581 gate.
- Train the read adapter per section 4; run the new `eval_read.py` and record the gates.
- Update `scripts/vllm_serve.sh` and `serve_gemma.sh` with the two new adapter entries and `--max-loras 4`; restart; `curl /v1/models` and confirm five names (base plus four adapters); run the UI smoke of one game turn and one reading.
- Push both adapters to the Hub, refresh the private dataset mirror, update `PROGRESS.md` (follow-up #8 closes with the write gate), regenerate this memo's final-numbers edition, commit, and push to master.

10. Immediate next steps after this memo

Work proceeds in the order this memo is written. First the reader dataset builder lands behind tests, then the teacher key is wired in and the dataset is generated and validated; the writer retrain and reader train share the single window; serving and UI cutover follow; evals and publication close the work. Nothing in the plan requires deleting data, re-scraping sources, or touching the locked game system prompt, and every step is idempotent in the house style of the repo: verify, record evidence in `PROGRESS.md`, and stop at clean boundaries. The end state is a game served by one always-on host where the corpse writes with weights tuned for continuation and reads with weights tuned for explication, each an exact match for the sentence it is asked to produce.

Typeset in Tinos 12pt - metrically identical to Times New Roman (Apache 2.0). Generated by `scripts/make_handoff_memo.py`; regenerate after the eval gates report final numbers.